

ITPROFEL2: OPEN SOURCE PROGRAMMING

Learning Module: Midterm - Week 8 (Loop Structures)

Instructor: Henry V. Dela Cruz

I. Overview and Objectives

This week, we expand on our introduction to control flow by dedicating our focus entirely to loop structures. You will learn how to iterate over sequences using `for` loops, build infinite conditional loops using `while`, and carefully manage iteration flow using loop control statements.

Intended Learning Outcomes (ILOs):

- Define the purpose of loops in software development.
- Differentiate between `while` loops (condition-based) and `for` loops (collection-based).
- Implement Loop Control Statements (`break`, `continue`, `pass`) to manage execution flow.
- Design interactive programs that loop indefinitely until a specific user command is received.

II. What is a Loop?

In programming, a loop is used to execute a block of code repeatedly. Instead of writing the same code ten times, you write it once and tell the computer to loop through it ten times. This adheres to the DRY (Don't Repeat Yourself) principle, a core tenet of efficient open-source development.

III. Types of Loops

1. The 'while' Loop

The `while` loop executes a set of statements **as long as a condition remains true**. It is generally used when you do not know beforehand how many times the loop needs to run (e.g., waiting for user input, or running a game loop).

```
# A simple while loop that acts as a countdown timer
countdown = 5

while countdown > 0:
    print(f"System launching in {countdown}...")
    countdown -= 1 # Decrement the counter to avoid an infinite loop

print("Liftoff!")
```

2. The 'for' Loop

A `for` loop is used for iterating over a sequence (that is either a list, a tuple, a dictionary, a set, or a string). It is used when you know exactly how many times the code block should execute.

```
# Iterating through a string
for letter in "BAKAWAN":
    print(letter)

# Using the range() function to loop a specific number of times
# range(3) will generate the numbers 0, 1, 2
for i in range(3):
    print(f"Processing batch {i}")
```

IV. Loop Control Statements

Loop control statements change the execution from its normal sequence. They allow you to skip iterations or break out of a loop entirely.

Statement	Description	Example Use Case
break	Terminates the loop statement entirely and transfers execution to the statement immediately following the loop.	Stopping a search algorithm once the target item is found.
continue	Causes the loop to skip the remainder of its body and immediately retest its condition prior to reiterating.	Skipping over invalid or corrupted data entries in a dataset while processing the rest.
pass	A null operation; nothing happens when it executes. Used as a placeholder in code blocks that are syntactically required but need no code yet.	Creating the skeleton of a complex if/else chain or loop before writing the actual logic.

```
# Using 'break' and 'continue' together
for num in range(1, 10):
    if num == 3:
        print("Skipping 3")
        continue # Jumps back to the top of the loop

    if num == 6:
        print("Breaking at 6")
        break # Kills the loop completely

    print(f"Number is {num}")
```

Laboratory Coding Session: The Interactive Guessing Game

Scenario: You are tasked with creating a simple CLI security mechanism. The user must guess a randomly predefined "Security Code" (a number between 1 and 20). The program must use a while loop to allow continuous guessing until the user either guesses correctly or types 'quit' to exit.

Instructions (Your code must implement these exact rules):

1. Define a constant variable `SECRET_CODE = 14`.
2. Set up a `while True:` loop to run continuously.
3. Inside the loop, use `input()` to ask: "Guess the security code (1-20) or type 'quit':".
4. First, check if the input is the string 'quit'. If it is, print "Security breach averted. Shutting down." and use the `break` statement to exit the loop.
5. If the input is not 'quit', cast the input to an integer.
6. Compare the integer to the `SECRET_CODE`.
 - If it matches, print "Access Granted." and break the loop.
 - If it does not match, print "Incorrect code. Try again." and let the loop reiterate.

Sample Program Run (Expected Output):

```
Guess the security code (1-20) or type 'quit': 5
Incorrect code. Try again.
Guess the security code (1-20) or type 'quit': 19
Incorrect code. Try again.
Guess the security code (1-20) or type 'quit': 14
Access Granted.
```

Submit your completed .py file once your terminal behaves exactly like the loop shown above.